

Afghanistan Girls Digital School

Aperçu de l'architecture technique (Technical Architecture Overview)

Document de synthèse préparé pour une évaluation académique — Département d'informatique, Université de Genève
• 23 juillet 2026

1. Contexte et mission

Afghanistan Girls Digital School (AGDS) est une application éducative mobile destinée aux adolescentes afghanes exclues de l'enseignement secondaire formel depuis 2021. Elle couvre dix matières du programme national (7^e à 12^e année) et dessert quatre profils d'utilisateurs distincts : élèves, parents, instructrices/instructeurs et administration de l'école, avec une interface disponible en dari, pashto, anglais et français.

2. Architecture technique

Le projet suit une architecture client-serveur classique, entièrement construite sur des services « sans serveur » (serverless) afin de minimiser les coûts d'exploitation et la charge de maintenance d'infrastructure — un choix pertinent pour un projet à but humanitaire opérant avec des ressources limitées.

Couche	Technologie	Rôle
Client mobile	Flutter / Dart, Riverpod, GoRouter	Interface Android/iOS, architecture « Clean Architecture » par module fonctionnel
API applicative	Cloudflare Workers, Hono (TypeScript)	Logique métier, authentification JWT, notation des examens et devoirs
Base de données	Cloudflare D1 (SQLite distribué)	Schéma relationnel versionné (38 migrations)
Stockage de fichiers	Cloudflare R2	PDF de cours, fichiers audio, photos de devoirs, avatars
Intelligence artificielle	LLM (RAG), Google Gemini Vision, Azure Cognitive Services	Tuteur virtuel, correction automatique de devoirs manuscrits, synthèse/reconnaissance vocale

Le tuteur IA repose sur une génération augmentée par récupération (RAG) « maison » : les leçons sont vectorisées et stockées dans D1, la similarité cosinus étant calculée directement dans le Worker, sans base vectorielle dédiée. La visioconférence des séminaires utilise Jitsi Meet, avec repli vers Cloudflare Stream ou un lien externe en cas d'indisponibilité.

3. Fonctionnalités principales et intégrité des données

- Parcours pédagogique à déverrouillage séquentiel des leçons, piloté côté serveur (le client ne fait qu'afficher l'état).
- Examens officiels et exercices d'entraînement notés automatiquement **côté serveur** — le score envoyé par le client n'est jamais utilisé tel quel, ce qui empêche la fraude (élément corrigé et durci lors de la dernière itération du projet).
- Devoirs manuscrits photographiés, corrigés par vision artificielle (OCR + notation + retour pédagogique en dari).
- Certificats numériques vérifiables publiquement par QR code, signés cryptographiquement (ECDSA P-256) pour empêcher toute falsification.
- Séminaires en direct, messagerie interne, système de notifications, tableau de bord parental et suivi de progression par matière.

4. Sécurité, fiabilité et gouvernance des données

- Authentification par jeton JWT ; mots de passe hachés par PBKDF2 (100 000 itérations) — bcrypt/argon2 indisponibles nativement dans l'environnement Workers.
- Limitation de débit (rate limiting) sur les routes d'authentification, contre les attaques par force brute.
- Journal d'audit (table *audit_logs*) pour les actions administratives sensibles (changement de note, invocation de l'IA, etc.).
- Restriction CORS au domaine officiel de l'école (aucune origine générique acceptée).
- Domaine applicatif dédié (plutôt qu'un sous-domaine générique du fournisseur cloud), pour éviter le filtrage réseau régional observé chez certains utilisateurs.

5. Échelle du projet

Métrique	Valeur
Fichiers Dart (client)	404
Lignes de code Dart	≈ 70 400
Fichiers TypeScript (serveur)	32
Lignes de code serveur	≈ 12 100
Migrations de base de données	38
Matières couvertes	10 (mathématiques, physique, chimie, biologie, anglais, littérature d'ici, histoire, géographie, éducation islamique, informatique)
Niveaux scolaires	7 ^e à 12 ^e année
Modules fonctionnels (client)	22 domaines (« features »), dont 12 dédiés à l'administration

6. Limites connues et travaux à venir

Par souci de rigueur académique, les limites suivantes sont documentées et suivies dans le dépôt du projet :

- **Absence de mode hors-ligne complet** — la synchronisation différée (file d'attente locale, cache des leçons) n'est pas encore implémentée ; il s'agit d'une limite critique compte tenu de la connectivité réseau instable dans une partie du public cible.
- **Couverture de tests automatisés quasi nulle** — le dossier de tests du client contient un seul fichier pour ≈ 70 000 lignes de code.
- **Chiffrement au niveau des colonnes non implémenté** — certaines données sensibles (téléphone, date de naissance, codes d'invitation) sont actuellement stockées en clair dans la base.
- **Aucun audit de sécurité externe formel** à ce jour ; les identifiants d'infrastructure (comptes Cloudflare, base D1) figurent dans le dépôt Git et devraient être migrés vers des secrets avant toute ouverture publique du dépôt.

7. Conclusion

AGDS est aujourd'hui un produit minimum viable (MVP) fonctionnel et déployé en production, avec plusieurs mécanismes d'intégrité déjà opérationnels : notation exclusivement côté serveur, signature cryptographique des certificats, journal d'audit, limitation de débit et domaine applicatif dédié. Un travail de durcissement reste nécessaire — synchronisation hors-ligne, chiffrement des données sensibles, couverture de tests — avant d'atteindre un niveau de maturité « entreprise ». Ce document vise à fournir une base de discussion technique concise pour toute évaluation, collaboration académique ou revue de code avec le Département d'informatique de l'Université de Genève.